Práctica 1.
Escena básica y modos de visualización.

1.1. Objetivos

El objetivo de esta práctica es familiarizarse con el entorno de desarrollo de Godot, el sistema de nodos y la creación de escenas 3D simples. Los estudiantes aprenderán a:

- Crear una escena 3D con geometría básica (cubo y pirámide).
- Aplicar distintos materiales a los objetos.
- Implementar cambios de modo de visualización usando entrada por teclado.
- Controlar la cámara mediante teclas.

1.2. Requisitos previos

Para realizar esta práctica no se requiere experiencia previa en motores de juego ni gráficos 3D, pero antes de hacerla debes asegurarte de:

- Haber instalado Godot Engine (versión 4 o superior).
- Conocer los conceptos básicos de programación (estructuras de control, funciones, clases).
- Descargar los archivos de script (extensión `.gd`) que se usarán en esta práctica, son los siguientes:
  - `ejes3D.gd`: script para dibujar ejes de coordenadas en la escena.
  - `piramide.gd`: script para crear una pirámide mediante código.
  - `camara_3d_orbital_simple.gd`: script para controlar una cámara orbital con teclado y ratón.

1.3. Actividades

En las subsecciones a continuación se detallan cada una de las actividades a realizar, son las siguientes:

1. Crear un nuevo proyecto Godot
2. Crear un cubo en la escena
3. Añadir ejes de coordenadas
4. Añadir cámara
5. Asignar materiales
6. Añadir luz
7. Crea una pirámide
8. Cambiar el material para que se vea rojo.
9. Controlar una cámara orbital

1.3.1. Crear un nuevo proyecto Godot

1. Abrir Godot Engine y seleccionar “Nuevo proyecto”.
2. Asignar un nombre y una carpeta de destino.
3. Crear el proyecto con el renderizador por defecto (Forward+).

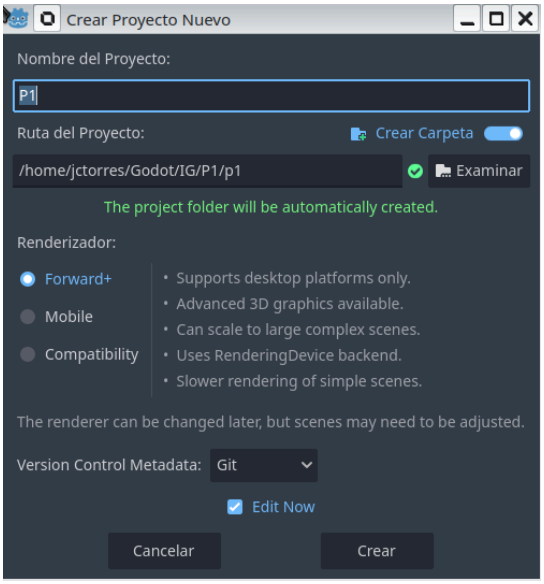


Figura 1: Crear proyecto

4. Abrir el editor y crear una nueva escena.
5. Añadir un nodo `Node3D` como nodo raíz, y renombrarlo a `EscenaPrincipal`.

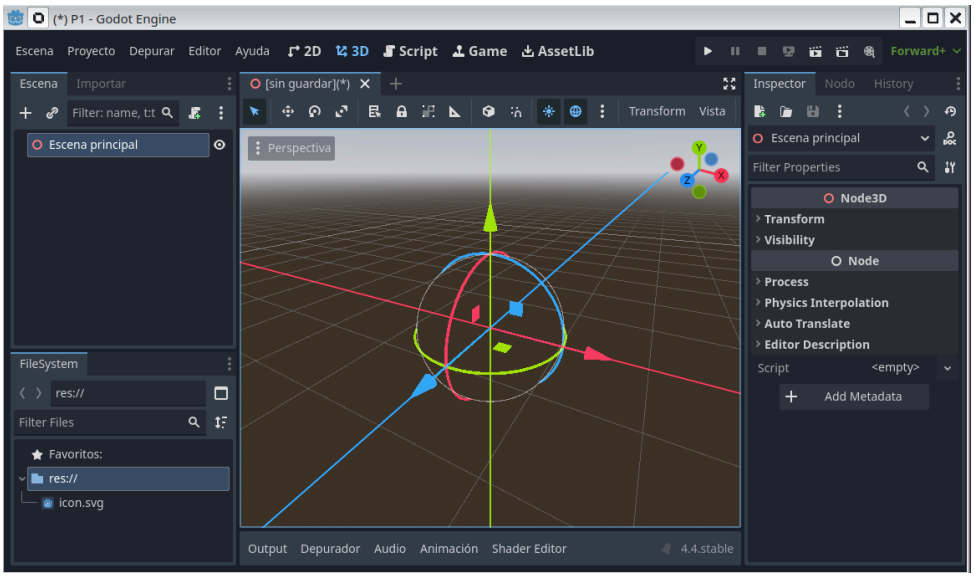


Figura 2: Crear escena

1.3.2. Crear un cubo en la escena

Se añadirá un objeto de tipo malla con forma de cubo, para ello se usará la clase `CubeMesh` de Godot, que ya está predefinida en el motor.

1. Añadir un nodo `MeshInstance3D` como hijo del nodo raíz.

2. Asigna una malla `CubeMesh` desde el panel Inspector (campo `Mesh`).
3. Renombrar el nodo como `Cubo`.
4. Moverlo a la posición `(0, 0.5, 0)`.

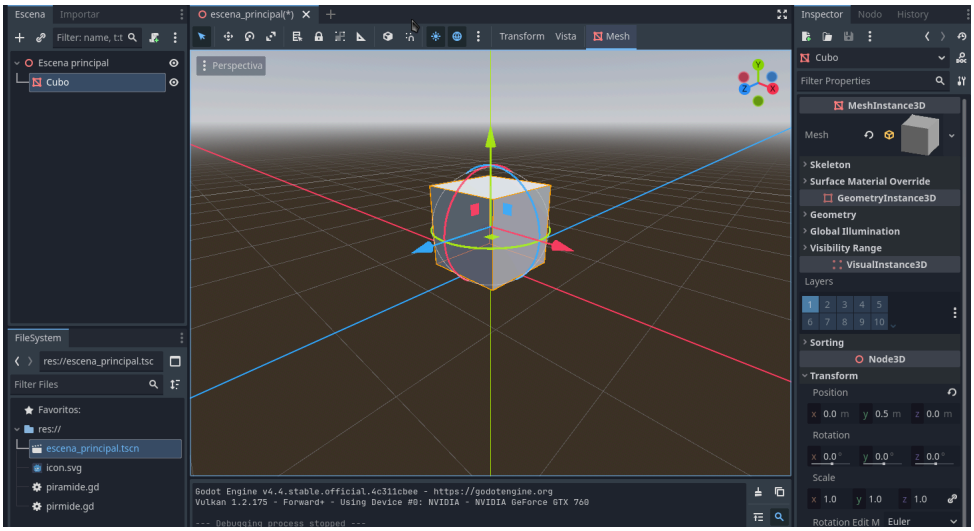


Figura 3: Añadiendo cubo

1.3.3. Añadir ejes de coordenadas

Se añade un nodo 3D con un script que al inicio genera mallas en los ejes de coordenadas, parecidas a las que aparecen en la vista 3D del editor, sirve para entender visualmente mejor la posición y tamaño de los objetos que se añaden.

1. Añadir un nodo de tipo `Node3D` y renombrarlo como `Ejes3D`.
2. Añadir a ese nodo un *script*, y al hacerlo, asociarle el archivo existente `ejes3D.gd`.

En la siguiente imagen se observan los ejes (tienen colores a los que no le afecta la iluminación)

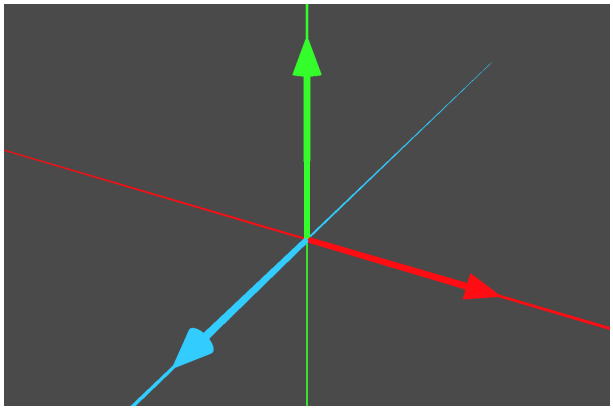


Figura 4: Ejes de coordenadas

1.3.4. Añadir cámara

Para poder visualizar la escena, es necesario añadir una cámara que define la posición y orientación del observador virtual que se usa para producir la imagen. Esta cámara es muy simple, luego se añadirá otra más completa.

1. Añadir un nodo `Camera3D` como hijo del nodo raíz.
2. Colocarlo en una posición adecuada para ver toda la escena, por ejemplo en `(1.5,1.5,2.0)` y hacer que la cámara mire al origen `(0,0,0)`. Esto se puede hacer añadiéndole el siguiente script a la cámara:

```
extends Camera3D
func _ready():
    position = Vector3( 1.5, 1.5, 2.0 )
    look_at( Vector3( 0.0, 0.0, 0.0 ), Vector3.UP )
```

3. Ejecutar.

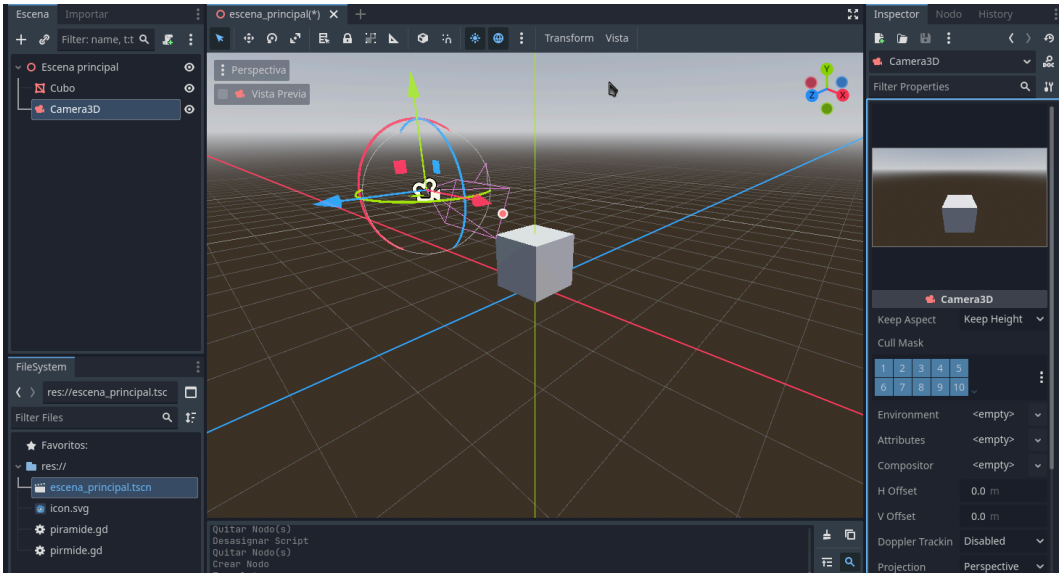


Figura 5: Cámara añadida a la escena

1.3.5. Asignar materiales

1. Con el cubo seleccionado, seleccionar `StandardMaterial3D` en la entrada `Surface Material Override` del panel `Inspector`.

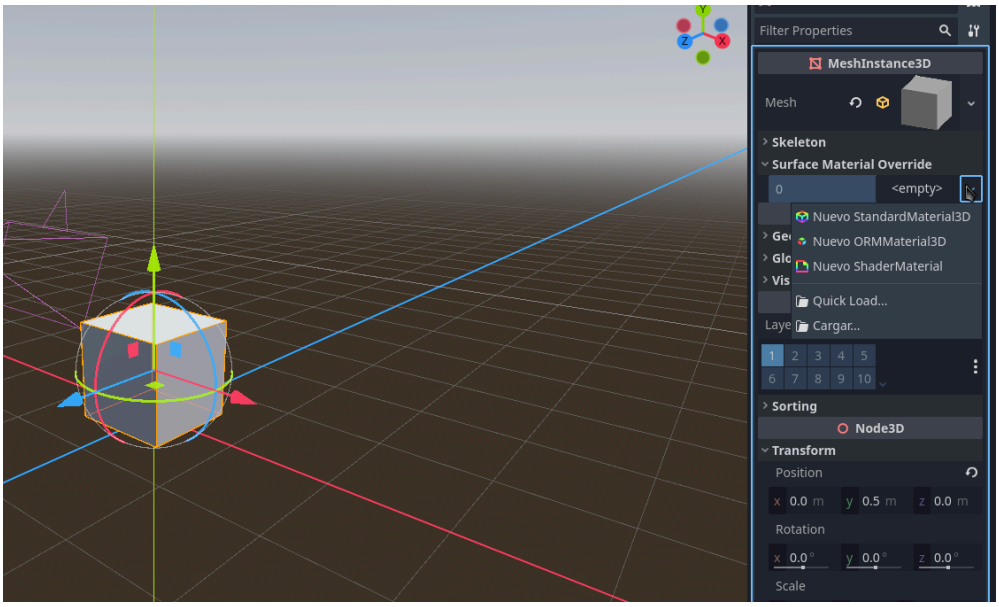


Figura 6: Asignación de material

2. Pulsar en la esfera y modificar el color del `Albedo`, asignar un color Amarillo.

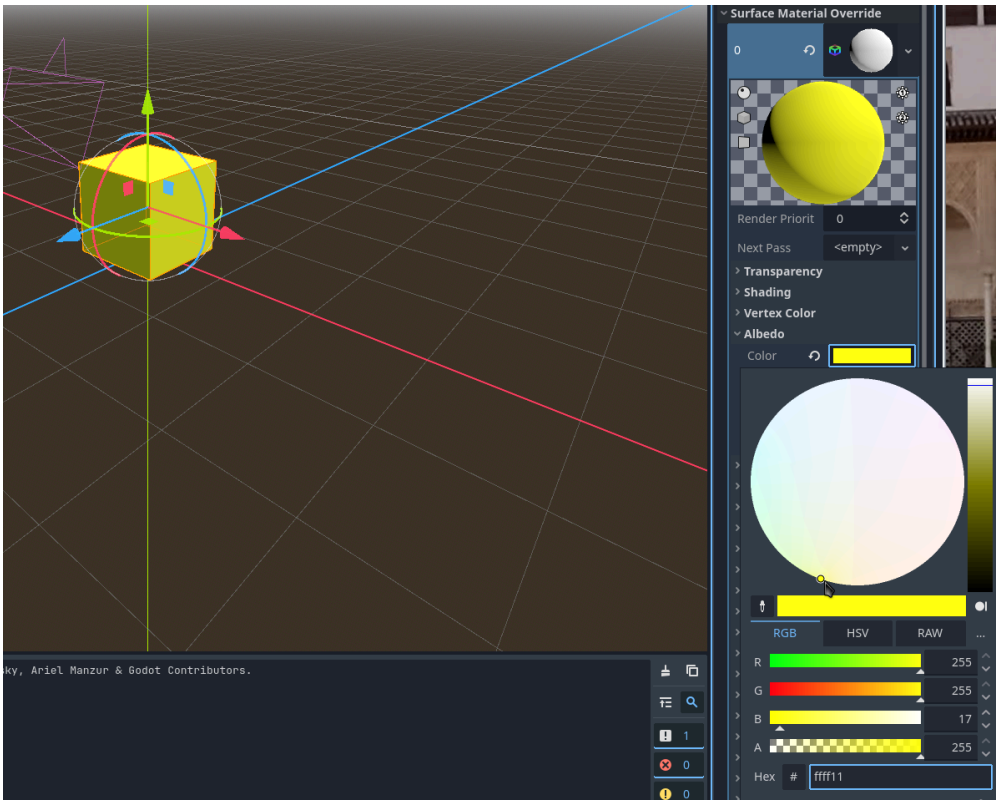


Figura 7: Cambiando el color

3. Ejecutar.

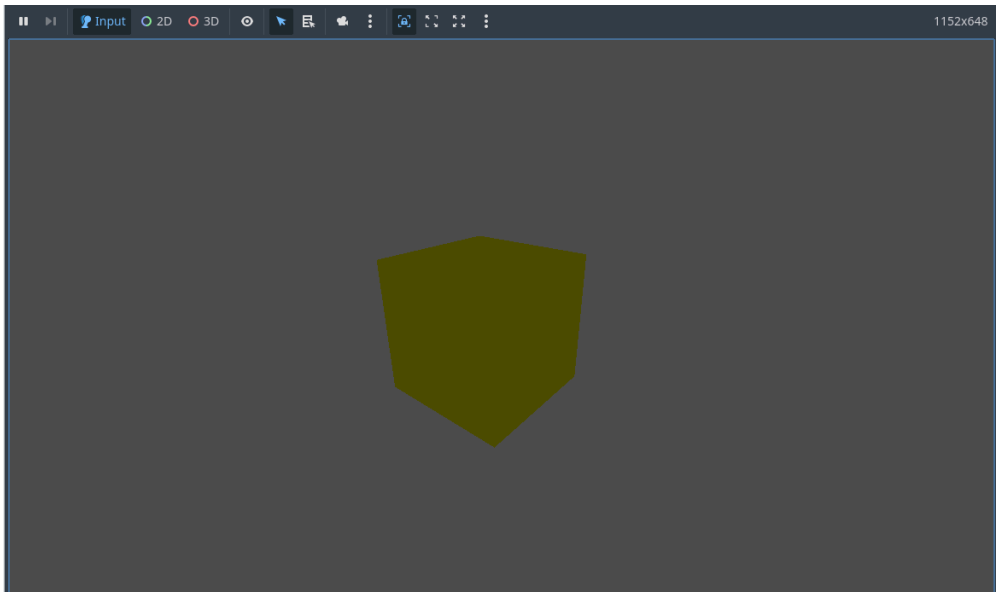


Figura 8: Ejecución tras añadir cámara

1.3.6. Añadir luz

1. Añadir un nodo `DirectionalLight3D` como hijo del nodo raíz.
2. Colocarlo fuera del origen para distinguir su representación de la del cubo
3. Rotarlo para que ilumine el cubo de forma que las tres caras tengan distinta tonalidad.

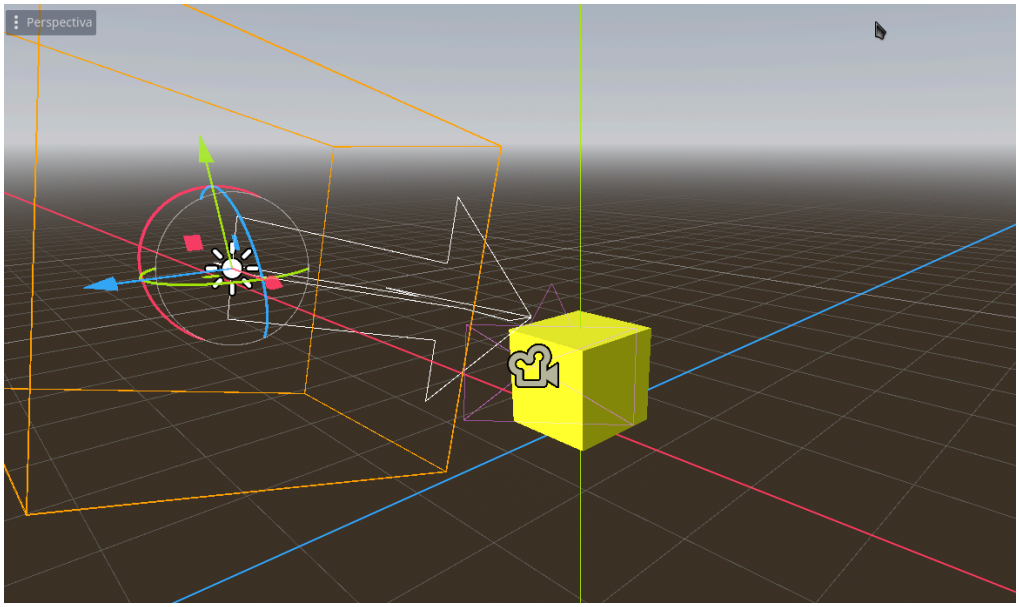


Figura 9: Fuente de luz direccional añadida a la escena

4. Ejecutar.

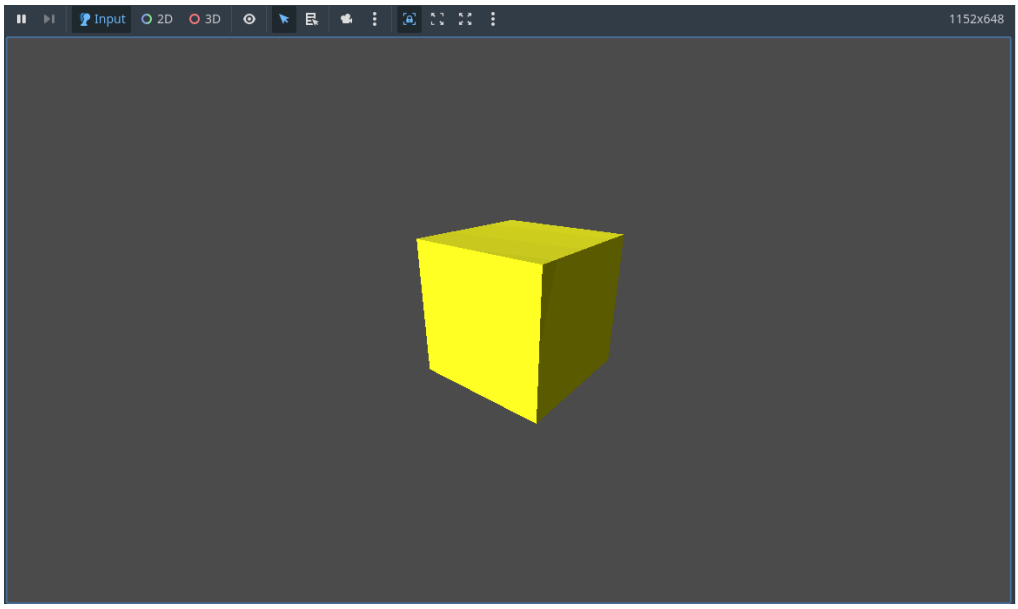


Figura 10: Ejecución con cámara y luz direccional

```gdscript
extends Node3D

@export var altura: float = 1.5

func _ready():
    var piramide = crear_piramide(altura)
    var mesh_instance = MeshInstance3D.new()
    mesh_instance.mesh = piramide
    add_child(mesh_instance)

func crear_piramide(h: float) -> ArrayMesh:
    var st = SurfaceTool.new()
    st.begin(Mesh.PRIMITIVE_TRIANGLES)

    # Coordenadas de la base (cuadrado centrado en el origen, lado 1)
    var p1 = Vector3(-0.5, 0, -0.5)
    var p2 = Vector3( 0.5, 0, -0.5)
    var p3 = Vector3( 0.5, 0,  0.5)
    var p4 = Vector3(-0.5, 0,  0.5)

    var apex = Vector3(0, h, 0)  # cúspide

    # Caras laterales (triángulos)
    _add_triangulo(st, p1, p2, apex)
    _add_triangulo(st, p2, p3, apex)
    _add_triangulo(st, p3, p4, apex)
    _add_triangulo(st, p4, p1, apex)

    # Base (dos triángulos)
    _add_triangulo(st, p1, p3, p2, Vector3.DOWN)
    _add_triangulo(st, p1, p4, p3, Vector3.DOWN)

    return st.commit()

# Añade triángulo y calcula normal automáticamente
func _add_triangulo(st: SurfaceTool,
                    a: Vector3, b: Vector3, c: Vector3,
                    normal_override: Vector3 = Vector3.ZERO):
    var normal = normal_override
    if normal == Vector3.ZERO:
        normal = Plane(a, b, c).normal
    st.set_normal(normal)
    st.add_vertex(a)
    st.add_vertex(b)
    st.add_vertex(c)
```

Figura 11: Código del script *para la pirámide de la práctica 1 (en el archivo* `piramide.gd`*)*

1.3.7. Crea una pirámide

Vamos a crear una pirámide de base cuadrada añadiendo la geometría mediante código.

1. Crear un nodo hijo de tipo `Node3D`.
2. Renombra el nodo como `Piramide`.
3. Asóciale el script con el código que aparece en la figura 11, y que tienes disponible en el archivo `piramide.gd`.

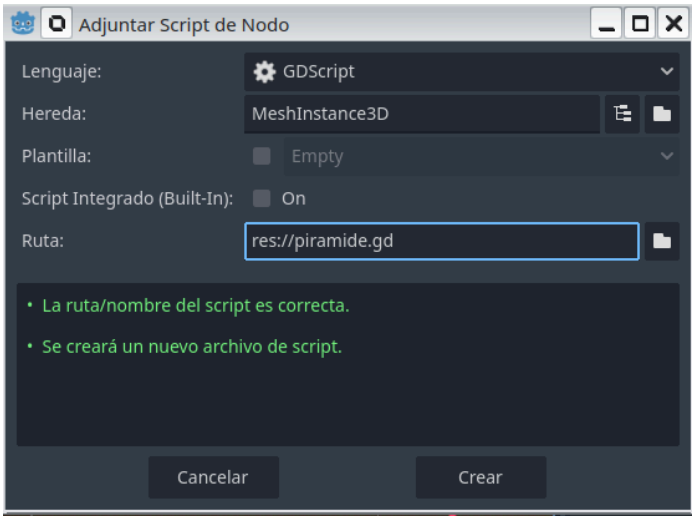


Figura 12: Creación de un script para la pirámide

4. Posiciona la pirámide sobre el cubo, en las coordenadas `(0,1,0)`.
5. Modifica el parámetro `altura` en el `Inspector` asignándole valor `0.5`.
6. Ejecuta.

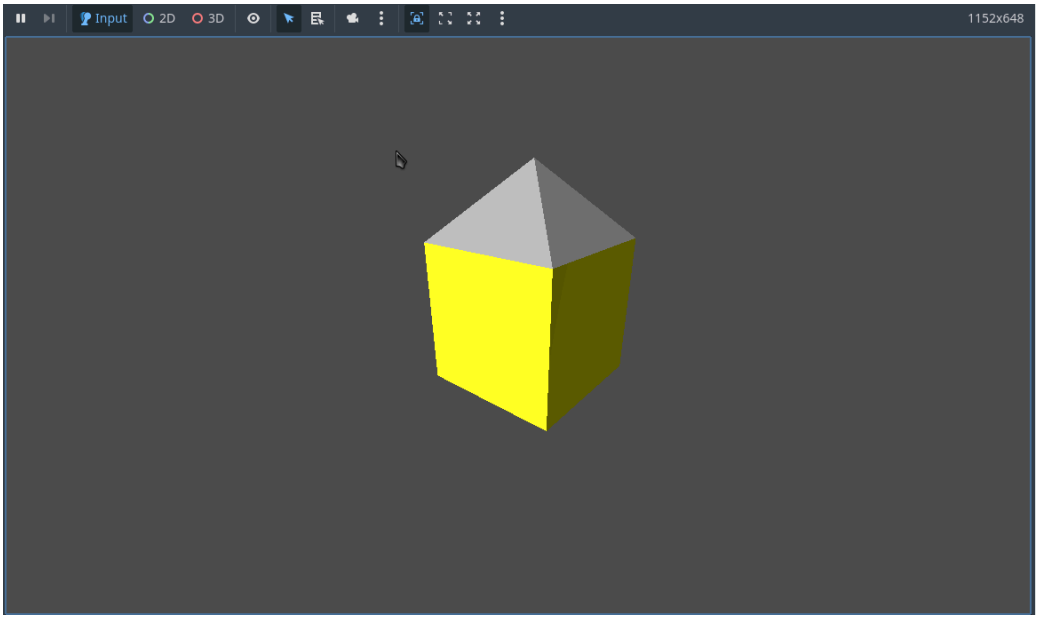


Figura 13: Ejecución con pirámide sobre el cubo

1.3.8. Cambiar el material para que se vea rojo.

Se le cambiará el material a la pirámide para que se vea de color rojo uniforme.

1. Edita la función `_ready()` del script anterior para añadir la descripión del material:

```
func _ready():
  var piramide = crear_piramide(altura)
  var mesh_instance = MeshInstance3D.new()
  var material = StandardMaterial3D.new()
  mesh_instance.mesh = piramide
  material.albedo_color = Color(1.0, 0.1, 0.2)  # Naranja claro
  mesh_instance.material_override = material
  add_child(mesh_instance)
```

2. Ejecuta

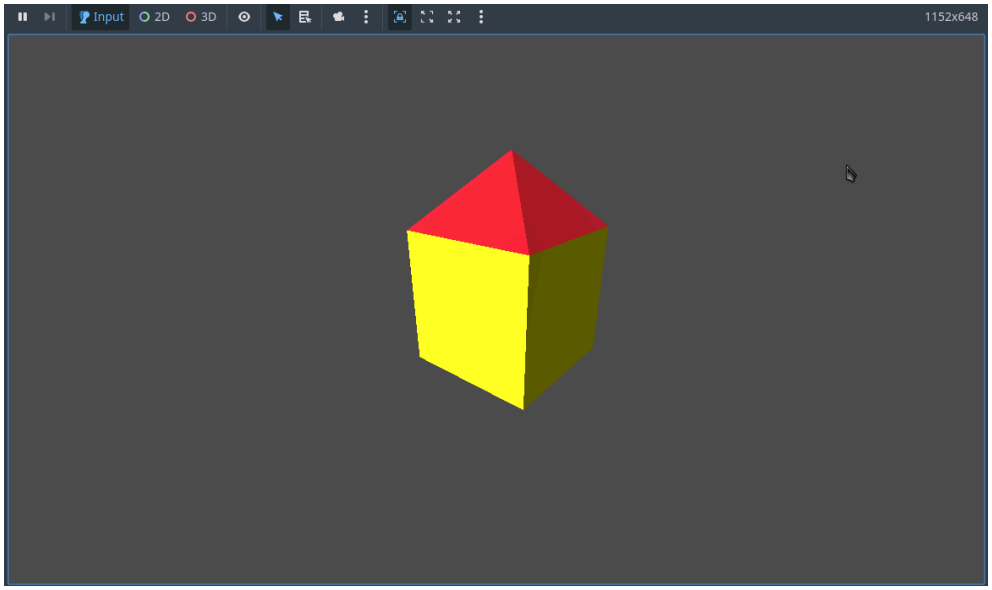


Figura 14: Ejecución con pirámide sobre el cubo

1.3.9. Controlar una cámara orbital por teclado y ratón

Vamos a hacer que la cámara pueda girar alrededor del origen para mostrar la escena desde diferentes posiciones, y que se pueda controlar con ratón además del teclado.

1. Borra el nodo `Camera3D` que se añadió antes. Debes borrar el nodo y el script asociado.
2. Añade un nuevo nodo `Camera3D` como hijo del nodo raíz, y renómbarlo como `Camara3DOrbital`.
3. Añade al nuevo nodo el script que está en el archivo `camara_3d_orbital_simple.gd`, que contiene el código de la figura 15.

Comprueba que puedes controlar la cámara con el teclado y el ratón. Puedes usar:

- Teclas de cursor para orbitar horizontal y verticalmente.
- Teclas +/-, o teclas de página anterio/siguiente o rueda del ratón para acercar/alejar la cámara.
- Botón derecho del ratón presionado y arrastrar para orbitar con el ratón.

```
extends Camera3D

const at   := 2.5   ## ángulo de rot. con teclas (en grados x pulsación)
const ar   := 0.5   ## ángulo de rot. con ratón (en grados x pixel)
var   bdrp := false ## botón derecho del ratón presionado sí/no
var   dz   := 3.0   ## distancia en Z de la cámara al origen
var   dxy  := Vector2( 0.0, 0.0 ) ## ángulos hor. y vert. (en grados)

func _actualiza_transf_vista(  ) -> void :
    var ahr  := ((45.0+float(dxy.x))*2.0*PI)/360.0 ## ang.horiz.radi.
    var avr  := ((30.0+float(dxy.y))*2.0*PI)/360.0 ## ang.vert.radi.
    var tras := Transform3D().translated( Vector3( 0.0, 0.0, dz))
    var rotx := Transform3D().rotated( Vector3.RIGHT, -avr )
    var roty := Transform3D().rotated( Vector3.UP, ahr )
    transform = roty*rotx*tras   ## actualiza transform del nodo cámara

func _ready() -> void :  ## se ejecuta una vez al inicio
    _actualiza_transf_vista() ## inicializa la vista al inicio

func _input( event : InputEvent ): ## procesa evento de entrada
    var av : bool = true ## actualizar vista sí/no

    if event is InputEventKey and event.pressed: ## teclas
        match event.keycode:
            KEY_UP:    dxy += Vector2( 0, -at )
            KEY_DOWN:  dxy += Vector2( 0, +at )
            KEY_RIGHT: dxy += Vector2( -at, 0 )
            KEY_LEFT:  dxy += Vector2( at, 0 )
            KEY_MINUS, KEY_PAGEDOWN, KEY_KP_SUBTRACT: dz *= 1.05
            KEY_PLUS, KEY_PAGEUP, KEY_KP_ADD: dz = max( dz/1.05, 0.1 )
            _: av = false
    elif event is InputEventMouseButton: ## botón ratón o rueda
        match event.button_index:
            MOUSE_BUTTON_RIGHT: bdrp = event.pressed ; av = false
            MOUSE_BUTTON_WHEEL_DOWN: dz *= 1.05
            MOUSE_BUTTON_WHEEL_UP:   dz = max( dz/1.05, 0.1 )
            _: av = false
    elif event is InputEventMouseMotion and bdrp: ## movim. ratón
        dxy += ar * Vector2( -event.relative.x, event.relative.y )
    else: ## otros tipos de eventos (no hace nada)
        av = false ## (no actualizar transf)

    if av: _actualiza_transf_vista( )
```

Figura 15: Código del script *para la cámara orbital (en el archivo `camara_3d_orbital_simple.gd`)*

1.4. Entrega de la práctica

- Subir la carpeta del proyecto `godot` con todos los archivos necesarios comprimido en un zip.
- Analiza los scripts y contesta el test de la práctica en Prado.